



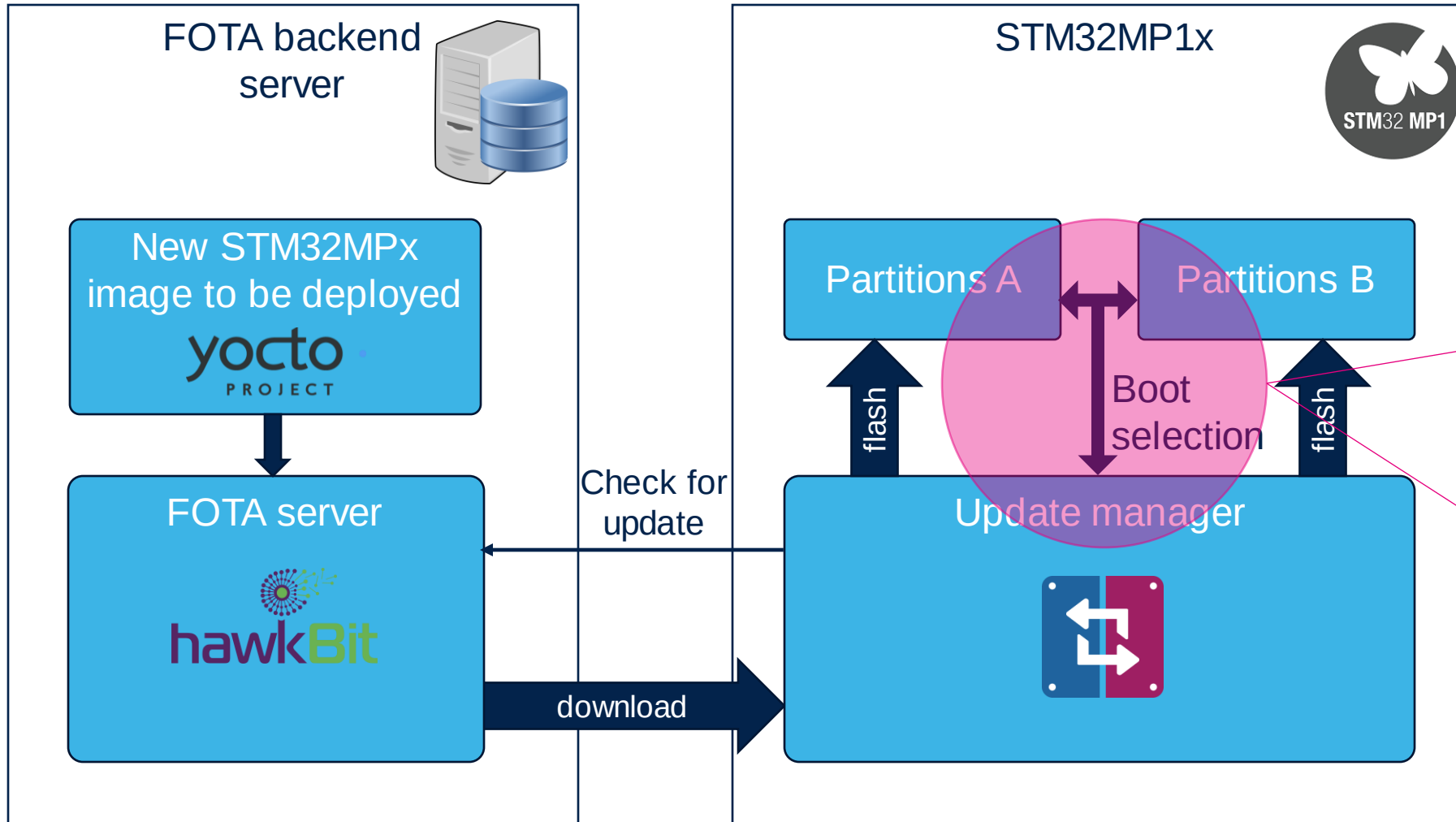
life.augmented

FOTA demo

A / B mechanism
flashlayout
Metadata partition

Christophe GUIBOUT

FOTA demo overview



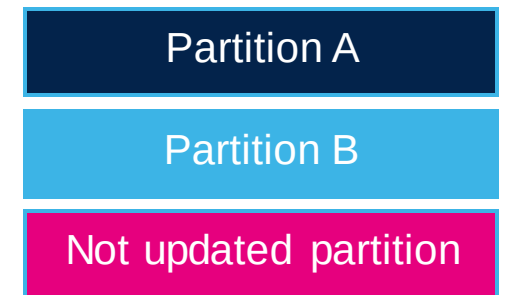
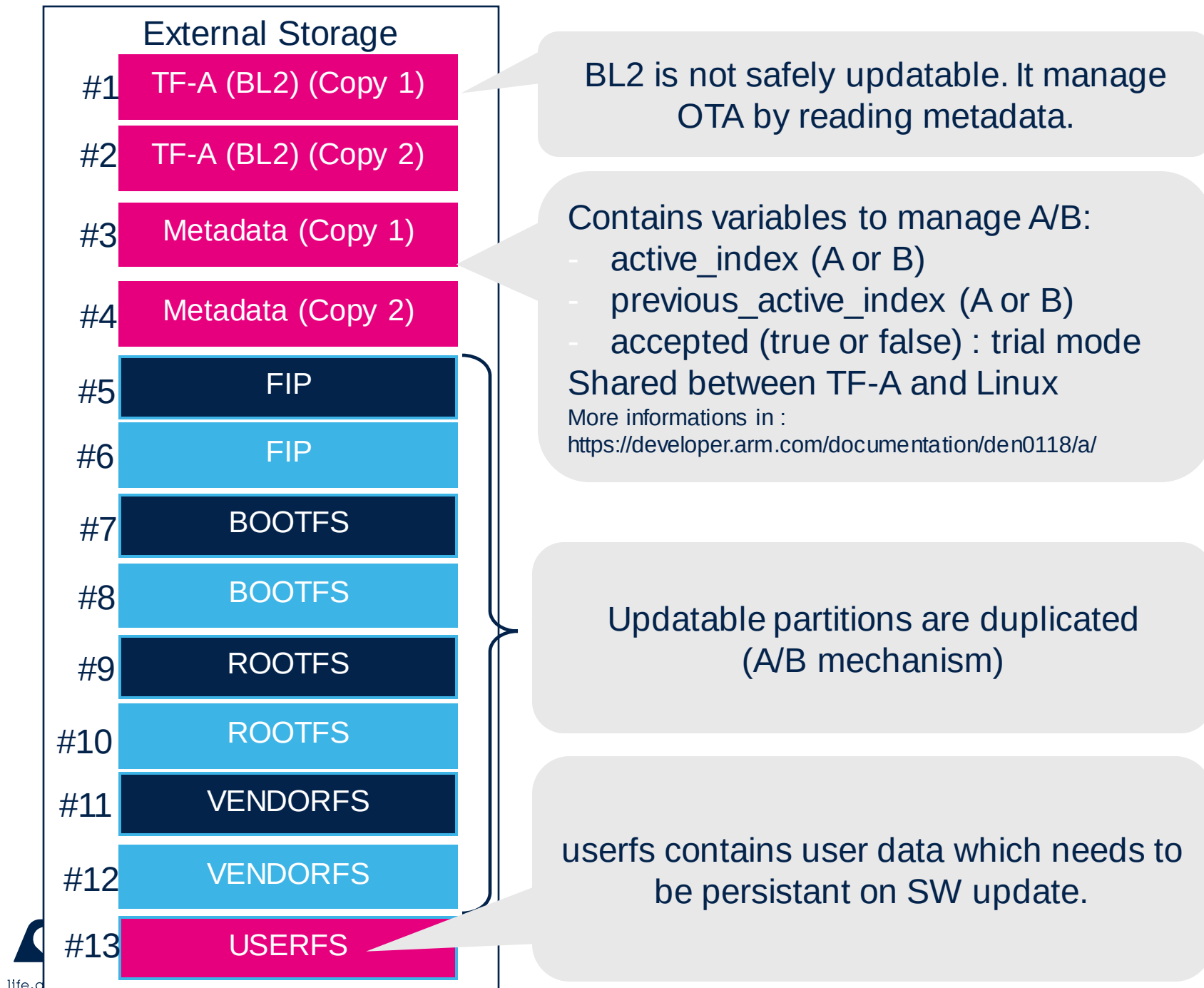
MMDV release:

- Boot selection mngt
 - New image failure mngt
 - Recovery to good known image
 - Only FIP and bootfs partitions are duplicated by default
- ➡ Supported by ST

FOTA demo:

- Also provides linux scripts to manage A/B variables (metadata)
- ➡ Provided as example

FOTA Layout A/B mechanism



- The metadata describes the status of the firmware process. It defines which partition (A or B) to use and stores the status of an update process.
- There are two update processes:
 - Nominal
 - TF-a loads good known partition
 - Trial
 - A new image has been flashed, but it has not been “accepted” as we don’t know if it is functional.
 - In trial mode, a counter is enabled and decrements at every reboot
 - On 1st reboot, tf-a boots on this new image :
 - If image boots till the end, update manager will mark this image as “accepted” in metadata, so the next reboot will be seen as a nominal boot.
 - If image boot failed, the system will reboot (on watchdog if a crash occurs, or requested by software if a problem is detected) : After 3 reboots (defined in tf-a), tf-a will boot the previous image.

Metadata partition structure example

Switching from A to B

This partition is updated by linux :

- After writing image into new partitions
- Before rebooting to new image

Content of the partition is read by tf-a (BL2) who selects on which fip (fip-a or fip-b) to boot.

```
{
  "metadata": {
    "crc_32": 1588275002,
    "version": 0,
    "active_index": 1,
    "previous_active_index": 0,
    "img_entry": {
      "img_0": {
        "img_bank_info": {
          "img_0_bank_0": {
            "accepted": true,
            "reserved": 101
          },
          "img_0_bank_1": {
            "accepted": false,
            "reserved": 102
          }
        }
      }
    },
    "configs": {
      "nb_fw_img": 1,
      "nb_fw_banks": 2
    }
  }
}
```

On the next reboot, the active partition will be B (A:0, B:1)

In case of failure of B image, the A image will be selected back after 3 attempts (defined in tf-a).

Partition A is considered as fonctionnal.

Partition B is in trial mode (not yet considered as fonctionnal) :

- If boot of Partition B succeeds, update manager service will update "accepted" to true at the end of B image boot.
- If not, after 3 Partition B boot failures, Partition A will be selected.

About the code : TF-A

A/B selection and trial counter

```
uint32_t plat_fwu_get_boot_idx(void)
{
    /*
     * Select boot index and update boot counter only once per boot
     * even if this function is called several times.
     */
    static uint32_t boot_idx = INVALID_BOOT_IDX;
    const struct fwu_metadata *data;

    data = fwu_get_metadata();

    if (boot_idx == INVALID_BOOT_IDX) {
        boot_idx = data->active_index;
        if (fwu_is_trial_run_state()) {
            if (stm32_get_and_dec_fwu_trial_boot_cnt() == 0U) {
                WARN("Trial FWU fails %u times\n",
                    FWU_MAX_TRIAL_REBOOT);
                boot_idx = data->previous_active_index;
            }
        } else {
            stm32_set_max_fwu_trial_boot_cnt();
        }
    }
    return boot_idx;
}
```

- Plat_fwu_get_boot_idx() is the key function in T-FA to manage A/B mechanism

Read metadata partition

If trial mode is selected in metadata, decrement the counter.

If the new image doesn't boot till the end after 3 attempts, TF-A will boot the previous image (previous_active_index).

About the code : TF-A

A/B index propagation through boot stages

- TF-A knows which FIP partition (fip-a or fip-b) to select thanks to metadata partition active_index value.
- Then, this information is propagated to U-boot through TAMP_BOOT_MODE_BACKUP_REG_ID tamper, so that u-boot can load the according kernel (bootfs-a or bootfs-b).

```
void plat_fwu_set_images_source(const struct fwu_metadata *metadata)
{
    ...
    boot_idx = plat_fwu_get_boot_idx();
    INFO("fwu active_index is %d\n", boot_idx);
    stm32_save_boot_mode((boot_idx == 1U) ? BOOT_RECOVERY : BOOT_NORMAL);
    ...
}
```

stm32_save_boot_mode() stores the value of active_index into TAMP_BOOT_MODE_BACKUP_REG_ID tamper, which will be used by u-boot.

About the code : U-boot

A/B index propagation through boot stages

```
arch/arm/mach-stm32mp/cpu.c
static void setup_boot_mode(void)
{
    ...
    u32 boot_ctx = readl(TAMP_BOOT_CONTEXT);
    u32 boot_mode =
        (boot_ctx & TAMP_BOOT_MODE_MASK) >> TAMP_BOOT_MODE_SHIFT;
    unsigned int instance = (boot_mode & TAMP_BOOT_INSTANCE_MASK) - 1;
    u32 forced_mode = (boot_ctx & TAMP_BOOT_FORCED_MASK);
    ...
}

static void setup_boot_mode(void)
{
    switch (forced_mode) {
        case BOOT_RECOVERY:
            env_set("preboot", "env set preboot; run altbootcmd");
            break;
        case BOOT_NORMAL:
            break;
    }
}
```

```
configs/stm32mp13_defconfig
CONFIG_BOOTCOMMAND="echo 'Boot A';setenv altboot 0;run bootcmd_stm32mp"
```

```
include/config_distro_bootcmd.h
"if test ${altboot} = 1;then setexpr distro_bootpart ${distro_bootpart} + 1; fi;"
```

```
include/configs/stm32mp13_common.h
"altbootcmd=echo 'Boot B';setenv altboot 1;run bootcmd_stm32mp\0"
```

According with TAMPER value (active_index), u-boot will load bootcmd (A) or altbootcmd (B)

altbootcmd will set \$altboot to 1, so in that case, the second bootable partition (bootfs-b) will be selected instead of the first one (bootfs-a) in case of normal boot. Warning : bootfs-a and bootfs-b must be contiguous.

About the code : Linux

Managed through scripts called by the update manager

- Here are key scripts to manage A/B mechanism provided in the demo (available in meta-st-ota):
 - post-install.sh
 - When : called by update-manager after flashing a new image into filesystem, but before switching to the new image
 - What : Update kernel cmdline (rauc.slot variable), bootfs and vendorfs mount points so that kernel running from bootfs-b loads vendorfs-b. Also configure metadata to select to correct active_index and set new image in "trial" mode.
 - st-status-mark-good.sh
 - When : service started by systemctl
 - What : returns a status to the update manager. In the example of the demo, it always returns good, but it can be customized.
 - st-boot-script.sh
 - When : Called by update manager at the end of the boot
 - What : configure metadata partition to new image as "accepted" which will disable trial mode.

About the code : rauc update manager

Bundle generation

- The bundle is an image built with Yocto to be stored on FOTA server and then to be flashed with update-manager.
- This file is rauc dependant
- The bundle is signed
- More details:
 - <https://rauc.readthedocs.io/en/latest/index.html>

meta-st-ota/recipes-core/bundles/update-st-bundle-stm32mp135f-dk.bb

```
inherit bundle
RAUC_BUNDLE_COMPATIBLE = "stm32mp1"
RAUC_BUNDLE_SLOTS = "fip bootfs vendorfs rootfs"
RAUC_BUNDLE_FORMAT = "verity"
RAUC_SLOT_bootfs = "st-image-bootfs"
RAUC_SLOT_bootfs[fstype] = "ext4"
RAUC_SLOT_bootfs[type] = "image"
RAUC_SLOT_bootfs[file] = "st-image-bootfs-openstlinux-weston-stm32mp1-ota.ext4"
RAUC_SLOT_rootfs = "st-image-weston"
RAUC_SLOT_rootfs[fstype] = "ext4"
RAUC_SLOT_rootfs[type] = "image"
RAUC_SLOT_rootfs[file] = "st-image-weston-openstlinux-weston-stm32mp1-ota.ext4"
RAUC_SLOT_vendorfs = "st-image-vendorfs"
RAUC_SLOT_vendorfs[fstype] = "ext4"
RAUC_SLOT_vendorfs[type] = "image"
RAUC_SLOT_vendorfs[file] = "st-image-vendorfs-openstlinux-weston-stm32mp1-ota.ext4"
RAUC_SLOT_fip = "tf-a-stm32mp"
RAUC_SLOT_fip[fstype] = "raw"
RAUC_SLOT_fip[type] = "boot"
RAUC_SLOT_fip[file] = "fip/fip-stm32mp135f-dk-optee.bin"
RAUC_SLOT_fip[rename] = "fip-stm32mp135f-dk-optee.bin"
RAUC_KEY_FILE = "${THISDIR}/files/development-1.key.pem"
RAUC_CERT_FILE = "${THISDIR}/files/development-1.cert.pem"
```

About the code : rauc update manager

Bundle download into A/B

- Update manager knows which partition is running thanks to kernel cmdline which contains "rauc.slot=A" or "rauc.slot=B".
- Update manager needs to know where to flash new downloaded image (bundle) :
In Rauc context, the file system.conf describes where to flash each partition depending on A/B mechanism.
- Example : If running image is A, update manager will flash rootfs from new downloaded image to mmcblk1p13.

meta-st-ota/recipes-core/rauc/rauc/system.conf

```
[system]
compatible=stm32mp1
bootloader=custom
mountprefix=/usr/local/rauc
max-bundle-download-size=805306368

[keyring]
path=/etc/rauc/ca.cert.pem

[handlers]
post-install=/usr/lib/rauc/post-install.sh
bootloader-custom-backend=/usr/lib/rauc/st-boot-script.sh

[slot.rootfs.0]
device=/dev/mmcblk0p12
type=ext4
bootname=A

[slot.rootfs.1]
device=/dev/mmcblk0p13
type=ext4
bootname=B

[...]
```

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented